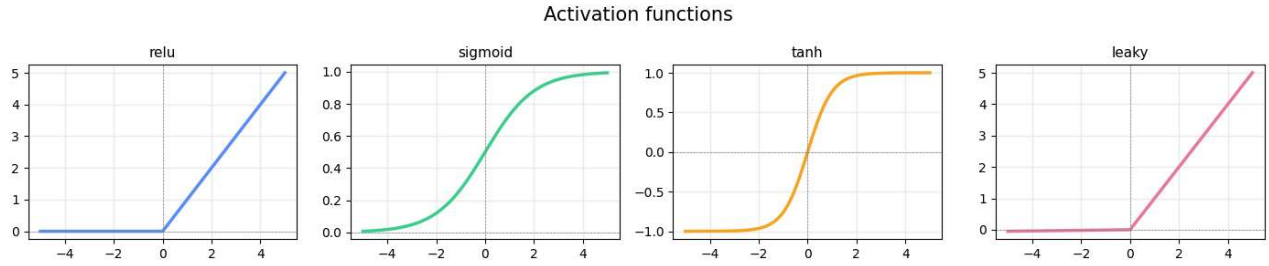
```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(-5,5,300)
relu= lambda x: np.maximum(0,x)
sigmoid= lambda x: 1/(1+np.exp(-x))
tanh= np.tanh
leaky= lambda x: np.where(x>0,x,0.01*x)
fig,axes=plt.subplots(1,4,figsize=(14,3))
fns=[relu,sigmoid,tanh,leaky]
names=['relu','sigmoid','tanh','leaky']
cols=["#5b8FF9",'#3ECF8E','#F5A623','#E879A0']
for ax,fn,name,col in zip(axes,fns,names,cols):
    ax.plot(x,fn(x),color=col,linewidth=2.5)
    ax.axhline(0,color='gray',lw=0.5,ls='--')
    ax.axvline(0,color='gray',lw=0.5,ls='--')
    ax.set_title(name,fontsize=11)
    ax.grid(alpha=0.2)
plt.suptitle('Activation functions',fontsize=15)
plt.tight_layout()
plt.show()
```




```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

#step:1 load MNIST

(x_train,y_train),(x_test,y_test)=keras.datasets.mnist.load_data()

#step:2 preprocess

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print(f'Train: {x_train.shape} Test: {x_test.shape}')

#step:3 build the model

model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128,activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(64,activation='relu'),
    layers.Dropout(0.2),

    layers.Dense(10,activation='softmax')
])

model.summary()
```

```
Train: (60000, 784) Test: (10000, 784)
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
dense_18 (Dense)	(None, 128)	100,480
dropout_12 (Dropout)	(None, 128)	0
dense_19 (Dense)	(None, 64)	8,256
dropout_13 (Dropout)	(None, 64)	0
dense_20 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

```
from IPython.core import history
#step:4 compile

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)

#step:5 callbacks - smart training

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau



early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose=1)

reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6,verbose=1)

#step:6 train

history=model.fit(
x_train,y_train,
epochs=50,
batch_size=64,
validation_split=0.1,
callbacks=[early_stop,reduce_lr],
verbose=1
)
```

```
Epoch 1/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.8810 - loss: 0.3953 - val_accuracy: 0.9628 - val_loss: 0.1258 - learn
Epoch 2/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9453 - loss: 0.1817 - val_accuracy: 0.9725 - val_loss: 0.0917 - learn
Epoch 3/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9586 - loss: 0.1379 - val_accuracy: 0.9758 - val_loss: 0.0799 - learn
Epoch 4/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9658 - loss: 0.1130 - val_accuracy: 0.9750 - val_loss: 0.0774 - learn
Epoch 5/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9695 - loss: 0.1008 - val_accuracy: 0.9785 - val_loss: 0.0730 - learn
Epoch 6/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9722 - loss: 0.0892 - val_accuracy: 0.9785 - val_loss: 0.0724 - learn
Epoch 7/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9747 - loss: 0.0779 - val_accuracy: 0.9790 - val_loss: 0.0693 - learn
Epoch 8/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9762 - loss: 0.0752 - val_accuracy: 0.9795 - val_loss: 0.0701 - learn
Epoch 9/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9790 - loss: 0.0674 - val_accuracy: 0.9800 - val_loss: 0.0718 - learn
Epoch 10/50
843/844 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9812 - loss: 0.0581
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9803 - loss: 0.0622 - val_accuracy: 0.9797 - val_loss: 0.0713 - learn
Epoch 11/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9848 - loss: 0.0473 - val_accuracy: 0.9817 - val_loss: 0.0617 - learn
Epoch 12/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9862 - loss: 0.0433 - val_accuracy: 0.9828 - val_loss: 0.0594 - learn
Epoch 13/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9864 - loss: 0.0422 - val_accuracy: 0.9822 - val_loss: 0.0620 - learn
Epoch 14/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9871 - loss: 0.0390 - val_accuracy: 0.9815 - val_loss: 0.0609 - learn
Epoch 15/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9874 - loss: 0.0383 - val_accuracy: 0.9848 - val_loss: 0.0572 - learn
Epoch 16/50
```

```
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9883 - loss: 0.0357 - val_accuracy: 0.9825 - val_loss: 0.0628 - learn
Epoch 17/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9889 - loss: 0.0336 - val_accuracy: 0.9822 - val_loss: 0.0638 - learn
Epoch 18/50
840/844 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9891 - loss: 0.0313
Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9887 - loss: 0.0328 - val_accuracy: 0.9825 - val_loss: 0.0653 - learn
Epoch 19/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9909 - loss: 0.0282 - val_accuracy: 0.9820 - val_loss: 0.0635 - learn
Epoch 20/50
844/844 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9917 - loss: 0.0264 - val_accuracy: 0.9835 - val_loss: 0.0616 - learn
Epoch 20: early stopping
Restoring model weights from the end of the best epoch: 15.
```

```
# ■■ Step 7: Evaluate on test set ■■■■■■■■■■■■■■■■■■■■■■■■
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f'Test Accuracy: {test_acc*100:.2f}%') # expect 97-98%
print(f'Test Loss: {test_loss:.4f}')

# ■■ Step 8: Plot training curves ■■■■■■■■■■■■■■■■■■■■■■■■
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Accuracy
ax1.plot(history.history['accuracy'], label='Train', color='#5B8FF9', lw=2)
ax1.plot(history.history['val_accuracy'], label='Val', color='#3ECF8E', lw=2)
ax1.set(title='Accuracy', xlabel='Epoch', ylabel='Accuracy')
ax1.legend(); ax1.grid(alpha=0.3)

# Loss
ax2.plot(history.history['loss'], label='Train', color='#F5A623', lw=2)
ax2.plot(history.history['val_loss'], label='Val', color='#E879A0', lw=2)
ax2.set(title='Loss', xlabel='Epoch', ylabel='Loss')
ax2.legend(); ax2.grid(alpha=0.3)
plt.tight_layout(); plt.show()
# Reading curves:
# GOOD: both curves decline and stay close together
# OVERFIT: train keeps dropping, val starts rising
# UNDERFIT: both stay high — model too simple or not trained enough
```

```
Test Accuracy: 98.20%
Test Loss: 0.0655
```